# Graduation Project Guidelines

# Table of Contents

Graduation Project Guidelines ..... 1
Table of Contents ..... 1
I. Introduction Chapter ..... 2
II. Literature Review Chapter ..... 3
III. Requirements Analysis and Specification Chapter ..... 4
IV. Design Chapter ..... 5
V. Implementation Chapter ..... 6
VI. Testing Chapter ..... 8
VII. Conclusion and Future Work ..... 8
VIII. Appendices ..... 9
IX. References ..... 9
References ..... 9

# I. Introduction Chapter

The first chapter in your GP report should be the introduction chapter; it is similar to the project proposal with only minor modifications. The introduction chapter should contain the following parts:

- **Introduction**

  Make sure that it includes all of the following things:
  - Provides a motivating brief background of the problem you are trying to solve.
  - Briefly discusses the related work that have been done to solve that problem and then discuss the technique/methodology you plan to use and how it is different from previous work.
  - Give an overview of the contents of the introduction chapter.

- **Problem Definition**
  - The problem definition is not the motivation, it is the technical challenge/problem you will face in order to achieve the aims and objectives of this project.
  - The term 'problem' means a **question** or issue to be examined.
  - You should **refine** the research ideas into a scientifically researchable questions.
  - **Specify** exactly what the research problem is.
  - Practice writing a good problem definition by looking into examples from published Master and PhD studies.

- **Aims and Objectives**

  The aim:
  - Should describe what you intend to achieve by doing this piece of work.
  - It is similar to the problem definition.

  The objectives:
  - They are the tangible outcomes that you should complete in order to achieve your aim.
  - They should be realistic, consistent, and link them to methods, timetable, and outcomes.

- **Project Timeline**
  - Cite the estimated time or time spans (months or weeks) for each phase of the project.
  - Usually as a Gantt Chart following the system development life cycle.

- **Team Qualifications**
  - It is like a mini-resume contained in the proposal.
  - Lists the work experience, similar projects, references, training, and education that shows familiarity with the project for each member of the team.

- **Conclusions**

Summarize the whole chapter concentrating on the following points:

- The importance of the problem you are studying.
- The previous attempts to solve the problem.
- The methodology you plan to use in order to solve the problem and how it is different from the previous related work?
- Mention that in the next chapter (Literature review) you will give a detailed explanation of the problem and the related work.

## II. Literature Review Chapter

The second chapter in your GP report should be the literature review chapter. This chapter should give the reader a clear understanding of the scientific background related to the problem you are studying, in addition to a clear overview of the work that have been done so far to solve that problem. The literature review chapter should be added after the Introduction chapter, and it should include the followings:

❖ **Introduction** (to the literature review chapter)

Make sure that it includes all of the following things:

- Links to the previous chapter, provides a recap of what has been presented.
- Explains the focus of this chapter. This chapter will… [This is important for the conclusion]
- Gives an overview of the contents of the literature review chapter.

❖ **Background**

When writing you should always think about your target audience. In your case the audience are professionals with a computing background, but not necessary specialists in the field of your project. This means that you should prepare them by providing some of all the following:

- The importance of the problem in this certain field.
- The history of the field that leads up to this problem.
- The definitions of fundamental concepts in the field.

After reading your background, the audience should have enough knowledge to understand and appreciate the efforts of other researchers you will discuss in the related work section, and also what you propose to achieve in this project.

❖ **Related work**

- Some projects are using a method ***x*** to solve a problem ***y***. Therefore, the related work should include papers that use ***x*** to solve ***y***. However, if there are none or only a few, then the related work should include:
    - Papers that used ***x*** to solve similar issues.
    - A comparison of applications/papers that solve ***y*** using other methods.
- Include up-to-date studies, and also important old ones.
- When conducting a literature review always start by summarizing the papers, highlight their contributions, methods, results and identify research gaps.

- Find a logical structure for presenting the papers, related work section should not only be summaries of papers, you should categorize the papers and discuss their similarities, differences and shortcomings. This logical structure could also be presented visually in a figure that illustrates the categories.
- Prepare the reader before you dive into the details of one of the categories discussed above, by providing a short paragraph explaining the category, also at the end of each category provide a closing comment to recap the information you provided.
- Having a comparison table of the related work is always a good idea, however make sure you have meaningful headers for the table, and that you are not comparing apples to oranges*. You can have multiple tables if your work requires it. If the tables span multiple pages, you can put them in the appendix, refer to them in the text, and provide a summary of the table. However remember tables must simplify the comparison, so don't overload them with lengthy text.

❖ **Conclusion** (to the literature review chapter)

- Summarize how you have achieved the focus of this chapter by addressing the following:
  - The main points in the background section.
  - The main points in the related work section, focusing on the research gap.
  - The importance of your work in targeting the research gap.
- Mention what will presented in the next chapter.

## III. Requirements Analysis and Specification Chapter

The specification provides the required system functionalities whereas the design provides a description of how to meet the specifications. A general method in software engineering projects is to start by defining user requirements (What does the user need from the system), describe the static architecture of the solution, identify sub-systems and modules and how are they inter-acting. Student should show his mastery of software engineering tools and methods by using UML diagrams. It is recommended that students read the software engineering book [1] to write correctly their project specification and design.

Students should describe the services/functionalities that a user requires from the system. Also, they should describe the constraints under which the system operates.

❖ **User Requirements**

First, describe how you arrived at the specified requirements (interviews, questionnaires, observations…). Then give the function and the characteristics of the system required by the target users as:

- ➢ Functional requirements: User functional requirements are described in a simple language and they are listed in this section.
- ➢ Non-Functional requirements: Define system properties and constraints. Non –

functional requirement are classified as following, see Figure 1.

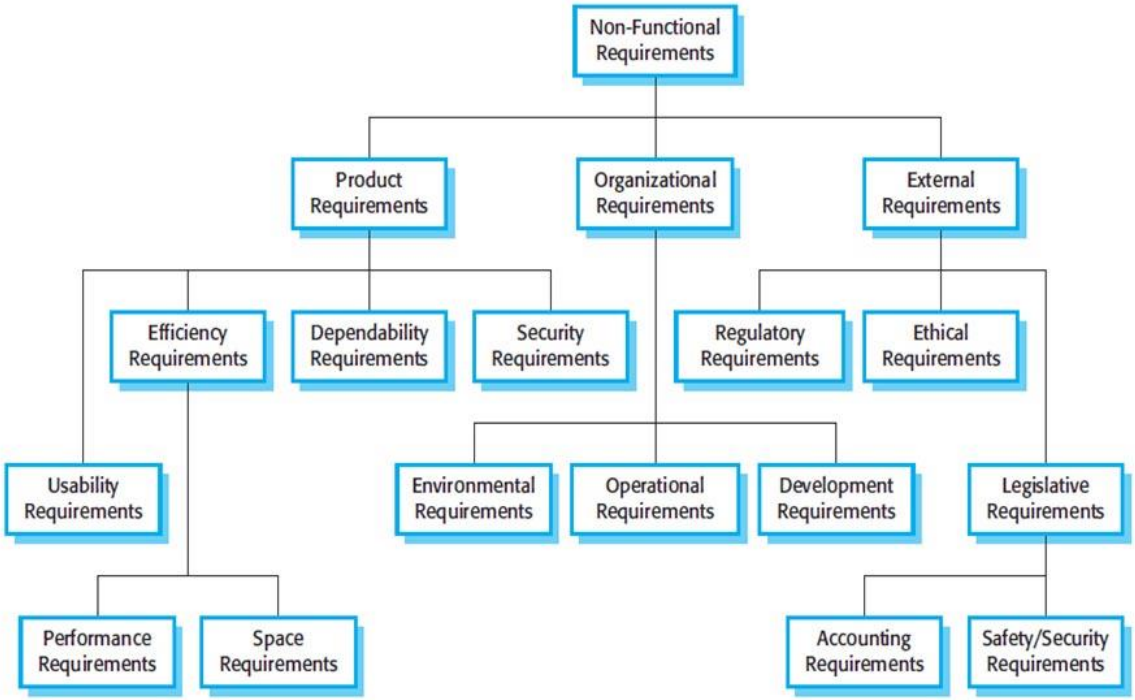


*Figure 1 :Non-Functional Requirements*

- **System Requirement**

Elaborate the user requirements to get a precise, detailed and complete version of them.

- Functional requirements: Use the use case diagram and use case descriptions.
- Non-Functional requirements: More details are involved compared to user non-functional requirements.

- **Project Management Plan**

Gantt Chart of the project plan must be supplied here. It should be drawn using a professional project management tool like MS Project.

## IV. Design Chapter

Student should describe the system structure and organization to fulfill the user requirement. Also, it is important to justify the design choices by explaining how the proposed design responds to the user requirements and why it is selected among other designs.

- **System Architecture:**

System architecture is presented using figures, diagrams, and textual explanations. You should give in a diagram your system sub-systems components and explain each sub-system functionality.

You should show how your subsystems are communicating (existent patterns; layered, client server, pipe, repository). System architecture could be composed of both hardware and software architecture.

- **Database Design**

Database design is provided using data dictionary, E/R diagrams…

- **Modular Decomposition**

Another structural level where sub-systems are decomposed into modules. Project modules are identified, and initial class diagrams are given in this section. Two modular decomposition models covered:

- Object-oriented decomposition is concerned with identifying object classes, their attributes and operations. When implemented, objects are created from these classes and some control model used to coordinate object operations. If you opted for Object Oriented Design (OOD), you should provide your classes and your subsystems object model.
- A pipeline or data-flow model where the system is decomposed into functional modules which transform inputs to outputs.

- **System Organization**

You should describe your system internal behavior. Described by diagrams such as; flow charts, Object state diagrams, Sequence diagrams, Data Flow Diagram. Flow chart and data flow diagram are used in functional models. Objects state diagrams, sequence diagrams and activity diagrams in object oriented models.

- **Algorithms**

You should include here your algorithms pseudocodes and describe them.

- **Alternative Designs/Methods**

Discuss the alternative designs and justify your design choice. This is mainly related to the system architecture, system decomposition and algorithms choices.

- **Graphical User Interface Design**
  Propose a design for your user interface.

## V. Implementation Chapter

The Implementation section is similar to the Specification and Design sections It describes the system, but it does in more detail, down to the code level. Do not explain all code in the system, and do not add more code in this section. Complete source code should be added separately on the

appendix section along with CD. Only the most important parts of your code (i.e., some methods, some classes, algorithms, etc.) should be listed and explained in more details, such as:

- Code that used for critical parts to the operation of the system. In other words, the most important code dealing with the project problems to achieve the objectives,
- Code that you feel might be of particular interest to the reader for some reason, and
- Code of your innovative way of implementing an algorithm, data structure, etc for achieving your project objectives listed in the Introduction chapter (your contributions in code).

List the name and references of all tools that were used in your project and explain why you select them to be parts of your project. Also, you have to mention any unexpected problems you encountered when implementing the system and how/what extent you solved them. Some of these common problems are: complexity of applying some algorithms, lack of the existence of system documentation, etc.

In the Implementation Chapter, please do not add flowcharts because they are already mentioned in the low level design subsection in the design section of your project. So, we have to distinguish between the flowchart and the implementation section.
You should describe your software requirements, the implementation of data structures and procedures. Also, explain the encountered problems and challenges and how are they resolved. The chapter may include the following subsection with an emphasis on the team innovation is important.

## A. Implementation Requirements

### ❖ Hardware Requirements

Give the required hardware specification and the reasons for the hardware choice.

### ❖ Software Requirements

Give the required software and the reasons for choosing them.

### ❖ Programming Language(s)

Give the reasons for your programming language(s) choice(s) and discuss the reuse of existing packages and components.

### ❖ Tools and Technologies

Give a state of the art and a **comparison of used tools and technologies in your project field.** Justify the use of development tools and implementation platforms. Demonstrate by a critical analysis how the used tools and technologies are appropriate for your project. Also give the tools used in your analysis and design chapter and justify their use.

## B. Implementation Details

- **Deployment and Installation**

Provide the hardware deployment steps and software installation steps.

- **Data Structures Description**

Give a brief description of your data structures implementation issues. Database implementation description should be provided in this section if there is any database.

- **Procedures Description**

Give a brief description of the implemented or used procedures/functions/methods. Could be provided in a table (but not mandatory). Small pieces of code could be included if necessary but should not be longer than a half page. Complete source code should be included as an Appendix.

- **Graphical User Interface Description**

# VI. Testing Chapter

In Testing Chapter, you have to check the correctness of your project. This process can be satisfied by writing a set of test cases to check that all project components, such as hardware, code, integration, GUI, and database are working well and as expected by the project team members. For this task, you can use different software testing techniques, such as:

- Code coverage testing: just test the critical requirements and tasks because it is very complex and time consuming testing technique.
- Condition testing: you can test the critical conditions (If Statement, Switch Statement, …) in your code, and
- Path testing: you can write test cases to test each single task from starting point to the ending point (result). May there are different paths to the endpoint.

In this chapter, we know that it is very hard to list all test cases for your entire project, but at least add one test case for each task you implemented in your project. Please show the error results, explain why, and explain how you overcame with these errors.

# VII. Conclusion and Future Work

The conclusions should summarize the overall work from problem definition, objectives and design and implementation issues. Also, you should justify how your solution meets the requirements and what are its principal shortfalls. In addition, you should add how your solution could be improved in future.

## VIII. Appendices

Contains any supplementary material that is of secondary interest to the reader.

## IX. References

Reference all:

- Recourses
- Claims
- Facts

Use the IEEE Citation style.

**Note that you have only a single Reference section for the whole report and it should grow as you progress in writing the remaining chapters.**

# References

[1] I. Sommerville, Software Engineering, 10th éd., Pearson, 2015.

.

.

.

.